



RAG contra la máquina

¿Responderás a mis preguntas?

Resumen: Generación Aumentada por Recuperación, eso es todo.
Ese es el objetivo de este proyecto.

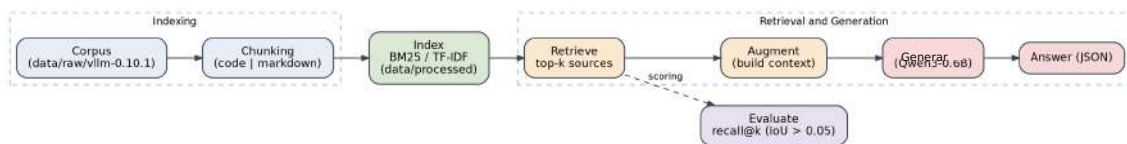
Versión: 2.0

Contenido

I Preámbulo	2
II Prólogo	3
III Instrucciones de IA	4
IV Introducción	7
V Instrucciones comunes	8
V.1 Reglas generales	8
V.2 Makefile	8
V.3 Directrices adicionales	9
V.4 Requisitos adicionales	9
VI Parte obligatoria	10
VI.1 Indexado del código base	10
VI.2 Recuperación	11
VI.3 Generación de respuestas	12
VI.4 Modelos de datos	12
VI.5 Salida	14
VI.6 Interfaz de línea de comandos	16
VI.7 Recorrido de punta a punta	16
VI.7.1 Estructura de proyecto esperada para la evaluación	16
VI.7.2 Ejecución de la canalización completa	17
VII Evaluación	19
VII.1 Métricas de evaluación	19
VII.1.1 Cálculo de Recall@k	19
VII.1.2 Rendimientos	20
VIII Requisitos del Readme	21
IX Parte adicional	23
X Entrega y evaluación entre pares	24
X.1 Instrucciones de recode	24

Capítulo I

Preámbulo



La tubería que vas a construir: ingerir e indexar un código base, recuperar los fragmentos más relevantes para una pregunta, entregarlos a un pequeño modelo local para generar una respuesta fundamentada y medir

la calidad de recuperación con `recall@k`.

Capítulo II

Prólogo

Un modelo de lenguaje está congelado en el tiempo. Todo lo que “sabe” quedó fijado el día en que terminó su entrenamiento. Pregúntale sobre una biblioteca publicada el mes pasado, un código privado o un informe de incidentes de ayer, y o bien se limitará a encogerse de hombros o inventará una respuesta segura de la nada.

Reentrenar un modelo cada vez que el mundo cambia es absurdamente costoso, así que hacemos trampa. En lugar de meter más conocimiento en el modelo, dejamos que busque la información correcta en el momento de responder, de la misma forma que no memorizas un manual entero, pero sabes qué página abrir.

Eso suena fácil hasta que lo intentas. El “manual” aquí es una base de código completa: miles de archivos, cientos de miles de líneas. Encontrar los dos o tres fragmentos que realmente responden a una pregunta, y solo esos, es un problema de aguja en un pajar. Si buscas muy poco, te pierdes la respuesta; si buscas demasiado, ahogas al modelo con ruido.

Nuestras intuiciones son malas a esta escala, pero buena ingeniería no lo es. Recuperar la evidencia adecuada de una montaña de datos, y luego hacer que una pequeña respuesta del modelo sea fiel a partir de ella, es de lo que trata este proyecto.

Capítulo III

Instrucciones de IA

● Contexto

Durante tu viaje de aprendizaje, la IA puede ayudar con muchas tareas diferentes. Tómate el tiempo para explorar las diversas capacidades de las herramientas de IA y cómo pueden apoyar tu trabajo. Sin embargo, siempre abórdalas con precaución y evalúa críticamente los resultados. Ya sea código, documentación, ideas o explicaciones técnicas, nunca puedes estar completamente seguro de que tu pregunta estuvo bien formulada o de que el contenido generado sea preciso. Tus compañeros son un recurso valioso para ayudarte a evitar errores y sesgos.

● Mensaje principal

- 👉 Utiliza la IA para reducir tareas repetitivas o tediosas.
- 👉 Desarrolla habilidades para generar indicaciones, tanto de codificación como de no codificación que beneficiarán tu futura carrera.
- 👉 Aprende cómo funcionan los sistemas de IA para anticipar y evitar mejor los riesgos comunes, sesgos y cuestiones éticas.
- 👉 Continúa desarrollando habilidades técnicas y de alto nivel trabajando con tus compañeros.
- 👉 Usa solo contenido generado por IA que entiendas completamente y por el que puedas responsabilizarte.

● Reglas del aprendiz:

- Debes tomarte el tiempo para explorar las herramientas de IA y entender cómo funcionan, para que puedas usarlas de forma ética y reducir posibles sesgos.
- Debes reflexionar sobre tu problema antes de generar indicaciones: esto te ayuda a escribir indicaciones más claras, detalladas y relevantes usando vocabulario preciso.

- Deberías desarrollar el hábito de revisar, evaluar, cuestionar y probar sistemáticamente cualquier cosa generada por IA.
- Siempre debes buscar la revisión de pares ☐ no confíes solo en tu validación.

● Resultados de la fase:

- Desarrolla habilidades de prompting tanto generales como específicas de dominio.
- Aumenta tu productividad con un uso eficaz de las herramientas de IA.
- Continúa fortaleciendo el pensamiento computacional, la resolución de problemas, la adaptabilidad y la colaboración.

● Comentarios y ejemplos:

- Normalmente te encontrarás con situaciones ☐ exámenes, evaluaciones y más ☐ donde debes demostrar un entendimiento real. Prepárate, sigue desarrollando tanto tus habilidades técnicas como interpersonales.
- Explicar tu razonamiento y debatir con tus pares a menudo revela lagunas en tu comprensión. Haz del aprendizaje entre pares una prioridad.
- Las herramientas de IA a menudo carecen de tu contexto específico y tienden a proporcionar respuestas genéricas. Tus pares, que comparten tu entorno, pueden ofrecer ideas más relevantes y precisas.
- Donde la IA tiende a generar la respuesta más probable, tus pares pueden aportar perspectivas alternativas y matices valiosos. Recurre a ellos como un punto de control de calidad.

✓ Buena práctica:

Le pregunto a la IA: “¿Cómo pruebo una función de ordenamiento?” Me da algunas ideas. Las pruebo y reviso los resultados con un compañero. Refinamos el enfoque juntos.

✗ Mala práctica:

Pido a la IA que escriba una función completa, la copio y pego en mi proyecto. Durante la evaluación entre pares, no puedo explicar qué hace ni por qué. Pierdo credibilidad y fracasa mi proyecto.

✓ Buena práctica:

Uso la IA para ayudar a diseñar un analizador. Luego recorro la lógica con un compañero. Detectamos dos errores y lo reescribimos juntos: mejor, más limpio y completamente entendido.

✗ Mala práctica:

Dejé que Copilot genere mi código para una parte clave de mi proyecto. Compila, pero no puedo explicar cómo maneja los pipes. Durante la evaluación, no logro justificar y fracaso mi proyecto.

Capítulo IV

Introducción

Nos encontramos con la llamada a funciones en `call_me_maybe`; esta vez el tema es RAG. Antes de mirar qué es RAG, concéntrate en lo que hace.

Un modelo solo “sabe” lo que fue entrenado; volver a entrenarlo para añadir conocimiento es lento y costoso. RAG toma otra ruta: en lugar de incorporar conocimiento al modelo, le das acceso a una fuente externa de tu elección y dejas que extraiga de ella en el momento de la respuesta.

En práctica, RAG tiene cuatro etapas:

- Indexación: organiza los datos para que puedan ser buscados.
- Recuperación: empareja una pregunta con el índice y extrae los fragmentos más relevantes.
- Aumentar: filtra esos fragmentos y colócalos en la ventana de contexto del modelo.
- Generación: lee ese contexto y produce la respuesta.



Lee todo el documento antes de escribir cualquier código: cada etapa alimenta a la siguiente.

Capítulo V

Instrucciones comunes

V.1 Reglas generales

- Tu proyecto debe estar escrito en Python 3.10 o más reciente.
- Tu proyecto debe adherirse al estándar de codificación flake8.
- Tus funciones deben manejar las excepciones de forma elegante para evitar fallos. Usa bloques try-except para gestionar posibles errores. Prefiere gestores de contexto para recursos como archivos o conexiones para asegurar la limpieza automática. Si tu programa falla debido a excepciones no manejadas durante la revisión, se considerará no funcional.
- Todos los recursos (p. ej., manejadores de archivos, conexiones de red) deben gestionarse correctamente para evitar pérdidas. Usa gestores de contexto cuando sea posible para un manejo automático.
- Tu código debe incluir anotaciones de tipo para parámetros de funciones, tipos de retorno y variables cuando corresponda (usando el módulo typing). Usa mypy para la comprobación estática de tipos. Todas las funciones deben pasar mypy sin errores.
- Incluye docstrings en funciones y clases siguiendo PEP 257 (p. ej., estilo Google o NumPy) para documentar el propósito, los parámetros y los retornos.

V.2 Makefile

Incluye un Makefile en tu proyecto para automatizar tareas comunes. Debe contener las siguientes reglas (lint obligatorio implica los indicadores especificados; se recomienda encarecidamente probar `-strict` para un mayor control):

- `install`: Instalar dependencias del proyecto usando `pip`, `uv`, `pipx` u otro gestor de paquetes de tu elección.
- `run`: Ejecutar el script principal de tu proyecto (p. ej., mediante el intérprete de Python).
- `debug`: Ejecutar el script principal en modo debug usando el depurador integrado de Python (p. ej., `pdb`).

- limpiar: Eliminar archivos temporales o cachés (p. ej., `__pycache__`, `.mypy_cache`) para mantener limpio el entorno del proyecto.
- lint: Ejecuta los comandos `flake8 .` y `mypy . --warn-return-any --warn-unused-ignores --ignore-missing-imports --disallow-untyped-defs --check-untyped-defs`
- lint-strict (opcional): Ejecuta los comandos `flake8 .` y `mypy . --strict`

V.3 Directrices adicionales

- Crea programas de prueba para verificar la funcionalidad del proyecto (no se envían ni califican). Usa marcos como `pytest` o `unittest` para pruebas unitarias, cubriendo casos límite.
- Incluye un archivo `.gitignore` para excluir artefactos de Python.
- Se recomienda usar entornos virtuales (p. ej., `venv` o `conda`) para aislar dependencias durante el desarrollo.

Si se aplican requisitos adicionales específicos del proyecto, se indicarán de inmediato debajo de esta sección.

V.4 Requisitos adicionales

Además de las reglas comunes de Python anteriores, este proyecto requiere:

- Tus modelos de datos deben usar `pydantic` para validación y seguridad de tipos: las estructuras que intercambias entre etapas, como las de la sección Modelos de Datos. Las clases de servicio u orquestación (indexador, recuperador, tubería, etc.) no tienen que ser `pydantic`.
- La norma de `flake8` se aplica también a archivos `bonus`.
- Debes usar `uv` como tu gestor de proyectos y paquetes: el revisor y la `moulinette` solo ejecutan `uv sync`.
- Tu sistema debe exponer una Interfaz de Línea de Comandos (CLI) construida con `Python Fire`, y barras de progreso (`tqdm`) para operaciones de larga duración.
- Debes usar el siguiente modelo:
 - **Qwen/Qwen3-0.6B (predeterminado)**
 - Puedes usar otros modelos siempre y cuando todo siga funcionando con `Qwen/Qwen3-0.6B`.
- Más allá de las herramientas requeridas arriba, puedes usar cualquier biblioteca que te guste.

Capítulo VI

Parte obligatoria

Construirás un sistema de Generación Aumentada por Recuperación que responda preguntas sobre un código base. Ingresas el repositorio vLLM proporcionado a un índice buscable, recuperas los fragmentos más relevantes para una pregunta, generas una respuesta a partir de ellos con Qwen/Qwen3-0.6B y mides la calidad de la recuperación con recall@k. Tu sistema se evalúa en si obtiene las ubicaciones de fuente correctas y produce respuestas basadas en ellas.

VI.1 Indexación del código base

Todo comienza con el índice. Lee los archivos que consideras útiles del repositorio vLLM enviado en los adjuntos, divídelos en fragmentos y guarda un índice que la recuperación pueda consultar en milisegundos. La indexación de todo el corpus debe tomar como máximo 5 minutos.

Un archivo de Python y una página Markdown no se partan de la misma manera, así que tu programa debe implementar dos estrategias de división distintas:

- Fragmentación de código Python,
- Fragmentación de Markdown / texto.

Para la recuperación en sí, implementa al menos uno de los dos métodos léxicos clásicos. La elección es tuya:

- TF-IDF,
- BM25.

Puedes explorar otros métodos además, siempre que se implemente alguno de estos dos.



El tamaño de los fragmentos es configurable a través de un argumento CLI (`--max_chunk_size`), con un valor predeterminado de 2000 caracteres. No lo superes: el moulinette rechaza cualquier fuente recuperada que exceda los 2000 caracteres (su `max_context_length`), y una única fuente demasiado larga hace que toda tu salida sea inválida. Los fragmentos más pequeños están bien; informa el efecto en tu `recall@k`.

```
Terminal - indexación

wil@42:~/rag$ uv run python -m src index --max_chunk_size 2000
Fragmentación:100%|#####| 1965/1965 [00:00<00:00, 16710 archivo/s]
Tokenización: 100%|#####| 13466/13466 [00:01<00:00, 10668 fragmento/s]
¡Ingestión completada! Fragmentos indexados bajo data/processed/
```

Indexando un corpus objetivo: el indexador divide cada archivo y persiste el índice bajo `data/processed/`.



Las preguntas y el código fuente rara vez usan las mismas palabras: una pregunta puede parafrasear una idea o citar un identificador tal como está. Lo que mantienes en el momento de indexar decide cuál de los dos aún puedes hacer coincidir.

VI.2 Recuperación

Con el índice construido, puedes buscarlo. Dada una pregunta, tu sistema devuelve los primeros `k` fragmentos más relevantes. Cada resultado es una ubicación de origen: un `file_path` y el rango de caracteres (`índice_de_primer_caracter`, `índice_de_último_caracter`) que abarca, con un ancho máximo de 2000 caracteres.



`file_path` debe coincidir exactamente con la ruta del corpus (p.ej. `data/raw/vllm-0.10.1/docs/features/lora.md`). El evaluador compara las rutas palabra por palabra, por lo que una ruta con un prefijo diferente nunca coincide.

La recuperación debe funcionar para una única consulta y en lote sobre un conjunto completo de preguntas leído desde JSON. En los conjuntos de datos de referencia, tu sistema debe alcanzar al menos `80% recall@5` en preguntas de documentos y `50%` en preguntas de código (la métrica se define en el capítulo de Evaluación).

```

Terminal - buscar

wil@42:~/rag$ uv run python -m src search "How to configure the OpenAI server?" --k 5
data/raw/vllm-0.10.1/examples/online_serving/openai_transcription_client.py [0:1352]
data/raw/vllm-0.10.1/examples/online_serving/prompt_embed_inference_with_openai_client.py [0:932]
data/raw/vllm-0.10.1/docs/deployment/frameworks/dstack.md [1936:3170]
data/raw/vllm-0.10.1/docs/models/supported_models.md [1699:3396]
data/raw/vllm-0.10.1/examples/online_serving/openai_chat_completion_client_with_tools.py [0:2000]

```

Una búsqueda de una sola consulta devuelve ubicaciones de origen clasificadas, cada una con su ruta de archivo y carácter span.

VI.3 Generación de respuestas

Con los fragmentos correctos recuperados, el sistema genera una respuesta en lenguaje natural utilizando Qwen/Qwen3-0.6B. Pasa el contexto recuperado al modelo dentro de su presupuesto de tokens y produce JSON estructurado siguiendo los modelos pydantic proporcionados.

Una respuesta satisfactoria es:

- Coherente y entendible,
- Basada en las fuentes recuperadas, sin grandes alucinaciones,
- Al punto: responde exactamente a la pregunta planteada.



El modelo obligatorio Qwen/Qwen3-0.6B tiene límites de razonamiento conocidos. Estos criterios describen el objetivo al que debe aspirar una respuesta, no una barra de aprobación/reprobación estricta: la calificación prioriza la calidad de recuperación, el anclaje y la estrategia de prompts sobre una redacción final perfecta. Una respuesta válida es coherente y principalmente basada en los documentos recuperados, incluso si está parcialmente incompleta debido a limitaciones del modelo base.

VI.4 Modelos de datos

Implemente los siguientes modelos pydantic; validan los datos intercambiados entre las etapas. El modelo MinimalSource representa una única fuente de información:

Modelo MinimalSource

```

class MinimalSource(BaseModel):
    ruta_archivo: str
    índice_del_primer_carácter: int
    índice_del_último_carácter: int

```

Los modelos UnansweredQuestion y AnsweredQuestion representan una pregunta sin respuesta y una pregunta respondida:

Modelos UnansweredQuestion y AnsweredQuestion

```
class UnansweredQuestion(BaseModel):
    id_pregunta: str = Campo(default_factory=lambda: "سازمان??")
    str(uuid.uuid4()))
    pregunta: str

class AnsweredQuestion(UnansweredQuestion):
    fuentes: Lista[MinimalSource]
    respuesta: str
```

El modelo RagDataset representa un conjunto de preguntas RAG: Modelo RagDataset

```
class RagDataset(BaseModel):
    preguntas_rag: Lista[AnsweredQuestion | UnansweredQuestion]
```

Los modelos MinimalSearchResults y MinimalAnswer representan los resultados de búsqueda y una respuesta:

Modelos MinimalSearchResults y MinimalAnswer

```
class MinimalSearchResults(BaseModel):
    id_pregunta: str
    pregunta: str
    fuentes_retiradas: Lista[MinimalSource]

class MinimalAnswer(MinimalSearchResults):
    respuesta: str
```

Los modelos StudentSearchResults y StudentSearchResultsAndAnswer representan resultados de búsqueda y resultados de búsqueda con respuestas:

Modelos StudentSearchResults y StudentSearchResultsAndAnswer

```
class StudentSearchResults(BaseModel):
    resultados_de_búsqueda: Lista[MinimalSearchResults]
    k: int

class StudentSearchResultsAndAnswer(BaseModel):
    resultados_de_búsqueda: Lista[MinimalAnswer]
    k: int
```

Los modelos proporcionados son una base. Puedes ampliarlos agregando nuevos modelos o campos extra (por ejemplo en el modelo de resultados de búsqueda) si tu implementación lo requiere.

VI.5 Salida

Cada comando escribe un archivo JSON que se ajusta a los modelos pydantic proporcionados:

- **Para operaciones de búsqueda: usa el modelo `StudentSearchResults` con:**
 - `search_results`: Lista de `MinimalSearchResults` que contienen `question_id`, `question` y `retrieved_sources`
 - `k`: Número de resultados solicitados
- **Para generación de respuestas: usa el modelo `StudentSearchResultsAndAnswer` con:**
 - `search_results`: Lista de `MinimalAnswer` que contiene `question_id`, `question`, `retrieved_sources` y `answer`
 - `k`: Número de resultados solicitados
- **Información de fuente: Cada `MinimalSource` contiene:**
 - `file_path`: ruta al archivo de origen, relativa a la raíz de tu proyecto y escrita exactamente como en el corpus ingerido (p. ej. `data/raw/vllm-0.10.1/...`); se compara palabra por palabra con la referencia
 - `first_character_index`: Posición del carácter inicial
 - `last_character_index`: Posición del carácter final

Formato de salida

La salida debe respetar la base mínima de los modelos proporcionados, pero puede mejorarse de la siguiente manera:

Ejemplo: salida de StudentSearchResults

```
{
  "search_results": [
    {
      "question_id": "q1",
      "question": "¿Cómo configurar el servidor de OpenAI?",
      "retrieved_sources": [
        {
          "file_path": "data/raw/vllm-0.10.1/docs/serving/openai_compatible_server.md",
          "first_character_index": 9867,
          "last_character_index": 10100
        },
        {
          "file_path": "data/raw/vllm-0.10.1/vllm/entrypoints/openai/api_server.py",
          "first_character_index": 267,
          "last_character_index": 400
        }
      ]
    }
  ],
  "k": 10
}
```

Para las respuestas, la salida debe seguir el modelo StudentSearchResultsAndAnswer:

Ejemplo: salida de StudentSearchResultsAndAnswer

```
{
  "search_results": [
    {
      "question_id": "q1",
      "question": "¿Cómo configurar el servidor de OpenAI?",
      "retrieved_sources": [
        {
          "file_path": "data/raw/vllm-0.10.1/docs/serving/openai_compatible_server.md",
          "first_character_index": 9867,
          "last_character_index": 10100
        },
        {
          "file_path": "data/raw/vllm-0.10.1/vllm/entrypoints/openai/api_server.py",
          "first_character_index": 267,
          "last_character_index": 400
        }
      ]
    }
  ],
  "answer": "Para configurar el servidor compatible con OpenAI en vLLM..."
}
```

VI.6 Interfaz de Línea de Comandos

Proporciona una CLI construida con Python Fire. Cada comando se invoca como `uv run python -m src <command> [options]`. Se requieren los siguientes comandos (las opciones mostradas son el mínimo; puedes añadir las tuyas):

- `index \u0013max_chunk_size <int>` Ingesta data/raw/ y construye el índice bajo data/processed/.
- `search <consulta> -k <int>` Devuelve las top-k fuentes para una sola consulta.
- `search_dataset \u0013dataset_path <ruta> \u0013k <int> \u0013save_directory <directorio>` Ejecutar búsqueda sobre un conjunto de datos completo y escribir un archivo JSON de StudentSearchResults.
- `answer <consulta> -k <int>` Responder a una sola consulta usando el contexto recuperado.
- `answer_dataset \u0013student_search_results_path <ruta> \u0013save_directory <directorio>` Generar respuestas para un conjunto de datos, produciendo un archivo JSON de StudentSearchResultsAndAnswer.
- `evaluate -student_search_results_path <ruta> -dataset_path <ruta>` Reporta tu recall@k respecto a un conjunto de datos de referencia, para tus propias pruebas.

Todos los caminos de entrada y salida deben ser argumentos configurables de la CLI y nunca estar codificados de forma fija. Incluye barras de progreso (tqdm) para operaciones prolongadas y maneja entradas degeneradas (consulta vacía, consulta sin sentido, k=0, archivos faltantes, JSON mal formado) con gracia. La CLI se prueba con tales casos límite y nunca debe fallar con una traza no controlada.



El comando `evaluate` es para tu propia iteración. El `recall@k` oficial utilizado durante la defensa se calcula con el ejecutable `moulinette` proporcionado, no por tu código. Tu solución nunca debe importar o llamar al `moulinette`.

VI.7 Recorrido de extremo a extremo

Esta sección muestra el flujo de trabajo completo, desde documentos en bruto hasta la evaluación y generación de respuestas, exactamente como lo ejecutan los scripts de examen de referencia.

VI.7.1 Disposición de proyecto esperada para la evaluación

Durante la defensa, la tubería de recuperación se ejecuta de extremo a extremo como un flujo automatizado único (`index search dataset moulinette evaluate_student_search_results`) usando guiones de examen de referencia. Los comandos individuales siguen disponibles para depuración, pero para hacer que la ejecución automatizada sea reproducible sin reestructuración manual, tu repositorio debe respetar la siguiente distribución y convenciones:

- src/: tu módulo de Python, ejecutable como `uv run python -m src <command>`.
- pyproject.toml y uv.lock en la raíz del repositorio, para que uv sync funcione desde la raíz.
- README.md en la raíz del repositorio.
- data/raw/: las fuentes indexadas (el repositorio vLLM proporcionado).
- data/processed/: el índice generado por el comando index.
- data/datasets/UnansweredQuestions/ y data/datasets/AnsweredQuestions/: los conjuntos de preguntas.
- data/output/search_results/<DatasetScope>/: la salida de search_dataset, segmentada por conjunto de datos.
- data/output/search_results_and_answer/<DatasetScope>/: la salida de answer_dataset, segmentada por conjunto de datos.

Todos los caminos de entrada y salida (`--dataset_path`, `--save_directory`, `--student_search_results_path`) deben ser argumentos CLI configurables y nunca deben estar codificados de forma fija: el evaluador los apunta a los conjuntos de datos y a directorios de salida dedicados. Si la estructura o la interfaz de comandos no se respeta, los scripts de referencia no pueden ejecutarse y las comprobaciones correspondientes se consideran fallidas por diseño.

VI.7.2 Ejecución de la tubería completa

La tubería se ejecuta con cuatro comandos, en ese orden: indexar el corpus, buscar un conjunto de datos completo, puntuar los resultados con el moulinette y luego generar respuestas. Los comandos de búsqueda y respuesta de una sola consulta mostrados anteriormente se comportan de la misma manera para una pregunta a la vez.

1. Indexar el corpus una vez:

```
uv run python -m src index --max_chunk_size 2000
Ingestión completa! Los índices se guardaron en data/processed/
```

2. Buscar en un conjunto de datos. Siempre limite --save_directory por conjunto de datos (UnansweredQuestions o AnsweredQuestions): los conjuntos de datos públicos comparten nombres de archivos, por lo que escribir cada ejecución en la misma carpeta sobrescribiría resultados anteriores.

```
uv run python -m src search_dataset
--dataset_path data/datasets/UnansweredQuestions/dataset_docs_public.json --k 10

--save_directory data/output/search_results/UnansweredQuestions
Guardados los resultados de búsqueda de estudiante en data/output/search_results/UnansweredQuestions/dataset_docs_public.json
```

3. Puntuar con el moulinette (renombrar moulinette-ubuntu/-fedora a moulinette primero). Los resultados del estudiante van primero, seguidos por el conjunto de AnsweredQuestions de referencia de verdad.

```
/moulinette evaluate_student_search_results
data/output/search_results/UnansweredQuestions/dataset_docs_public.json
data/datasets/AnsweredQuestions/dataset_public.json
--k 10 --max_context_length 2000
```

```
Los datos del estudiante son válidos: Verdadero
Resultados de la Evaluación
=====
Recall@1: 0.450 Recall@3: 0.590 Recall@5: 0.650 Recall@10: 0.720
```



Estas cifras ilustran solo el formato de salida. No son una puntuación de referencia. Los umbrales que debes alcanzar se definen en el capítulo de Evaluación.

4. Genera respuestas a partir de los resultados de búsqueda:

```
uv run python -m src answer_dataset --student_search_results_path
data/output/search_results/UnansweredQuestions/dataset_docs_public.json --save_directory
data/output/search_results_and_answer/UnansweredQuestions Cargadas 100 preguntas ... Procesadas 100 de 100
preguntas Guardado student_search_results_and_answer a .../UnansweredQuestions/dataset_docs_public.json
```

Capítulo VII

Evaluación

VII.1 Métricas de evaluación

La calidad de recuperación se mide con una métrica de recall@k.

VII.1.1 Cálculo de Recall@k

Para cada pregunta, recall@k es la proporción de sus fuentes correctas que recuperas en tus resultados top-k. Una fuente correcta se cuenta como encontrada cuando uno de tus resultados está en el mismo archivo y se superpone con su rango de caracteres.

La barra de superposición es baja (un IoU de 0,05), por lo que no es necesario coincidir exactamente con el tramo de referencia: recuperar un fragmento que cubra la región correcta del archivo correcto es suficiente. Un resultado en un archivo diferente nunca cuenta, por lo que la ruta del archivo debe ser exacta.

```
Terminal - evaluation

wil@42:~/rag$ ./moulinette evaluate_student_search_results \
  search_results/dataset_docs_public.json \
  datasets/AnsweredQuestions/dataset_docs_public.json --k 10
Student data is valid: True
Número total de preguntas: 100
Número total de preguntas con fuentes de estudiantes: 100

Resultados de Evaluación
=====
Preguntas evalúan      d: 100
Recall@1: 0.620 (62.0%)
Recall@3: 0.770 (77.0%)
Recall@5: 0.880 (88.0%)  <- docs threshold is 80%
Recall@10: 0.890 (89.0%)
```

La moulinette valida tu salida, luego informa recall en varios valores de k. Los documentos deben alcanzar un recall@5 del 80%, código 50%.

VII.1.2 Rendimientos

Tu sistema debe respetar rendimientos mínimos, enumerados a continuación:

- Tiempo de indexación: como máximo 5 minutos para todo el corpus.
- Rendimiento de recuperación: como máximo 90 segundos para 200 preguntas.
- **Recall@5: al menos 80% en preguntas de documentos y 50% en preguntas de código.**

Capítulo VIII

Requisitos de Readme

Se debe proporcionar un archivo README.md en la raíz de tu repositorio Git. Su objetivo es permitir que cualquier persona no familiarizada con el proyecto (pares, personal, reclutadores, etc.) entienda rápidamente de qué trata el proyecto, cómo ejecutarlo y dónde encontrar más información sobre el tema.

El README.md debe incluir al menos:

- La primera línea debe estar en *itálicas* y leerse: Este proyecto ha sido creado como parte del plan de estudios 42 por <login1>[, <login2>[, <login3>[...]]].
- Una sección Descripción que presente claramente el proyecto, incluida su meta y una breve visión general.
- Una sección Instrucciones que contenga cualquier información relevante sobre compilación, instalación y/o ejecución.
- Una sección Recursos que enumere referencias clásicas relacionadas con el tema (documentación, artículos, tutoriales, etc.), así como una descripción de cómo se utilizó la IA — especificando para qué tareas y qué partes del proyecto.

➡ **Pueden requerirse secciones adicionales dependiendo del proyecto (por ejemplo, ejemplos de uso, lista de características, opciones técnicas, etc.).**

Cualquier adición requerida se listará explícitamente a continuación.

Para este proyecto, el README.md también debe incluir:

- Arquitectura del sistema: Describe los componentes de tu canalización RAG y cómo interactúan
- Estrategia de división: Explica tu enfoque para la segmentación de documentos
- Método de recuperación: Detalla el algoritmo de recuperación y el mecanismo de clasificación

- Análisis de rendimiento: discuta las puntuaciones recall@k y el rendimiento del sistema
- Decisiones de diseño: explique las opciones de implementación clave
- Desafíos enfrentados: documente las dificultades encontradas y las soluciones
- Uso de ejemplo: proporcione ejemplos claros de cómo ejecutar su sistema



Su README debe estar escrito en inglés.

Capítulo IX

Parte de bonificación



La bonificación se evalúa solo una vez que se valida toda la parte obligatoria: cada requisito obligatorio cumplido y el sistema robusto incluso cuando un revisor lo empuja en direcciones extrañas. Hasta que la parte obligatoria se valide por completo, la bonificación no se evalúa en absoluto.

Hay cinco bonificaciones, cada una vale un punto. Todas se ejecutan en una máquina de campus solo con CPU, y cada una amplía el sistema obligatorio en una dirección que volverás a encontrar más adelante. Una bonificación cuenta solo si está implementada y funcionando, no solo descrita en el README.md, y es posible que te pidan demostrarla.

1. Embeddings semánticos: añade un índice vectorial construido con un modelo de CPU ligero (como all-MiniLM-L6-v2) junto a tu índice léxico.
2. Recuperación híbrida: combina las clasificaciones léxicas y semánticas en una única lista de resultados.
3. Indexación incremental: cuando un archivo cambia, vuelve a indexar solo ese archivo en lugar de reconstruir todo el índice.
4. Caché: almacena en caché el índice y los resultados de las consultas para acelerar el inicio en frío y consultas repetidas.
5. API HTTP local: expone la consulta del índice y respuestas a preguntas a través de una pequeña API HTTP local, para que el sistema pueda ser manejado por algo distinto de la CLI.

Capítulo X

Presentación y evaluación por pares

Presenta tu tarea en tu repositorio Git como de costumbre. Solo el trabajo dentro de tu repositorio será evaluado durante la defensa. No dudes en revisar dos veces los nombres de tus archivos para asegurar que son correctos.

Tu repositorio debe contener:

- directorio src/ con tu implementación
- pyproject.toml y uv.lock para la gestión de dependencias
- un Makefile que exponga las reglas de las Instrucciones Comunes (instalar, ejecutar, depurar, limpiar, lint)
- README.md con documentación completa
- Cualquier archivo de configuración adicional necesario para ejecutar tu solución



No incluyas archivos de datos grandes, pesos del modelo o salidas generadas en tu repositorio. El evaluador generará estos durante el proceso de evaluación. La pila de aprendizaje profundo y los pesos del modelo pueden sumar varios gigabytes, así que construye y ejecuta tu proyecto desde un lugar con suficiente espacio libre en disco.

Instrucciones de recodificación X.1

Durante la evaluación, ocasionalmente puede solicitarse una modificación breve del proyecto. Esto podría implicar un cambio menor de comportamiento, unas pocas líneas de código para escribir o reescribir, o una función fácil de agregar.

Aunque este paso puede no ser aplicable a todos los proyectos, debes estar preparado para ello si se menciona en las pautas de evaluación.

Este paso tiene como objetivo verificar tu comprensión real de una parte específica del proyecto. La modificación puede realizarse en cualquier entorno de desarrollo que elijas (p. ej., tu configuración habitual) y debería ser factible en unos pocos minutos, a menos que un plazo específico esté definido como parte de la evaluación.

Por ejemplo, se te puede pedir que realices una pequeña actualización de una función o guion, que modifiques una pantalla o que ajustes una estructura de datos para almacenar nueva información, etc.

Los detalles (alcance, objetivo, etc.) se especificarán en las pautas de evaluación y pueden variar de una evaluación a otra para el mismo proyecto.